Partial submission (issues with handling of 571 midterm, wanting to get something submitted)

Shani “Dill” De Silva, U44033199

EC 551 Advanced Digital Design with Verilog and FPGAs

Fall 2026, Prof. Douglas Densmore

Homework #2

Due: 10/20

1. Find a comparison chart of FPGAs (e.g., within product families is fine; Altera is an older FPGA maker acquired by Intel in 2015 that you might want to use). Highlight four differences between devices and discuss how these might be either beneficial for a design or detrimental to a design. Discuss these in terms of performance, power, and cost. In addition, briefly discuss the metrics by which the FPGAs are compared (e.g. the columns in the chart perhaps).

I have chosen to compare the Agilex® 5 FPGA E-Series 005B (B15A) and the Agilex® 5 FPGA E-Series 065A (B32A), which are in the same series but different enough that they should be interesting to compare.

Difference in LUTs per FPGA: the 005B has 17100 and the 065A has 222400. Both chips have the same ratio of LUTs to logic registers, but the 065A is a far larger FPGA which is helpful for larger designs.

Difference in Maximum Embedded Memory: the 005B has 3.06Mb on-chip, and the 065A has 38.25Mb available. The difference in memory correlates to a difference in power and cost.

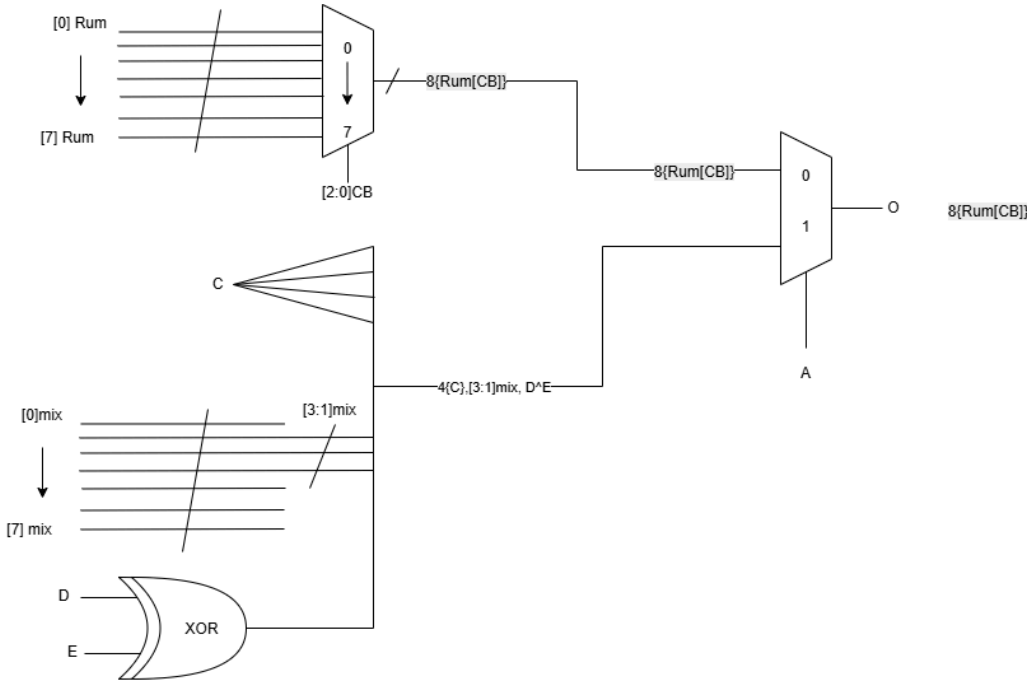
Difference in Fabric Speed Grades: the 005B has a speed grade of 4, 5, 6 while the 065A has a speed grade of 1, 2, 3. Smaller numbers are faster - the 065A has a higher performance. This is useful, especially when coupled with the fact that the 065A will likely be used to run larger synthesised code.

Difference in Peak INT 8 (TOPS): This refers to the theoretical maximum number of 8 bit operations per second, divided by 1 trillion. The 005B has 1.7, and the 065A has 25.99, which is an incredibly large difference in performance.

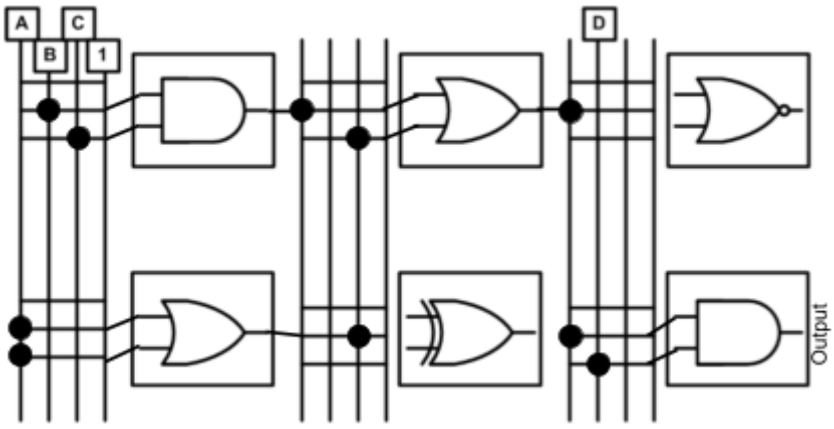
The metrics provided seemed to mainly affect size/area (changing costs) and performance, which is probably to be expected as I chose to compare two FPGAs from the same company and same release date. I didn’t find much information on power usage.

2. Draw a schematic showing what the following Verilog code is describing.

```
reg [7:0] O, mix, Rum;
reg [2:0] CB;
reg A, C, D, E;
assign O = A? {{4{C}}, mix[3:1], D^E} : {8{Rum[CB]}};
```



3. a. Connect (draw lines from gate inputs and outputs and/or add dark circles on the switch fabric) the primary inputs, gate inputs, and gate outputs to the provided FPGA switch fabric to realize the function: **(A+BC)D**



b. Assuming this design has only six 2-input LUTs and the six illustrated 4x4 crossbars, how many bits of SRAM would be needed to store connection and LUT information? (show work, state assumptions)

6 LUTS * 3 pins per LUT * 4 possible connections per pin = 72 bits
6 LUTS * 2 inputs ^2 = 24 bits (LUT outputs)
72+24=96 bits

I am assuming that a LUT cannot pass information backwards in this diagram.

c. How realistic is this FPGA? What is it missing (if anything)? How would you make this better? What would be some challenges of making this FPGA?

Not very realistic - quite small! Also, the LUTs are pre-defined as gates, instead of allowing me to program them myself. A way to improve this setup would be to add vertical wires, to make it easier to pass A to the next layer of LUTs. A challenge of making this FPGA would be that the gates come pre-decided, so it isn’t really an FPGA?

4. Create a circuit that counts the number of 1’s in a 4-bit value. The input should be 4-bits wide and the output should be a 3-bit binary number representing the value.

Provide verilog code, schematics, and relevant synthesis messages for all four parts.

a. Write a structural Verilog description of the design that synthesizes.

```
module countones (
    input  [3:0] four_in,
    output wire [2:0] three_out
);
wire and_01, and_23; //for the [2] digit
and(and_01, four_in[0], four_in[1]);
and(and_23, four_in[2], four_in[3]);
and(three_out[2], and_01, and_23);


wire xor_01, xor_23; //for the [0] digit
xor(xor_01, four_in[0], four_in[1]);
xor(xor_23, four_in[2], four_in[3]);
xor(three_out[0], xor_01, xor_23);


//wire and_01, and_23 reused from calculation for [2] digit.
wire and_02, and_03, and_12, and_13;
wire or_allpairs, nand_0123;
```

Partial submission (issues with handling of 571 midterm, wanting to get something submitted)
Shani “Dill” De Silva, U44033199

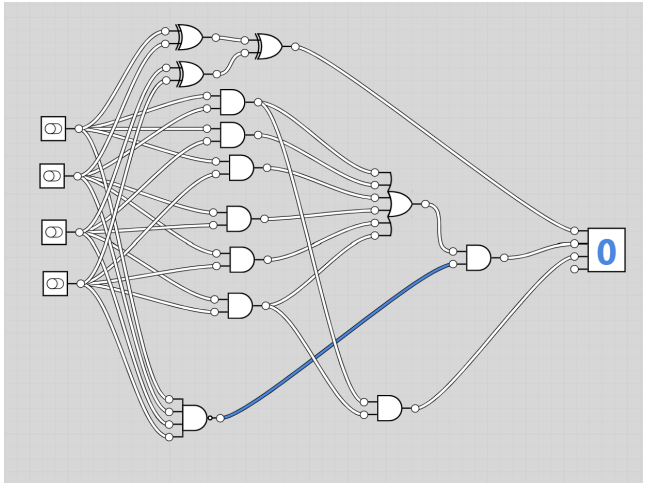
```
//and(and_01, four_in[0], four_in[1]); already calculated
and(and_02, four_in[0], four_in[2]);
and(and_03, four_in[0], four_in[3]);
and(and_12, four_in[1], four_in[2]);
and(and_13, four_in[1], four_in[3]);
//and(and_23, four_in[2], four_in[3]); already calculated

or(or_allpairs, and_01, and_02, and_03, and_12, and_13, and_23);
nand(nand_0123, four_in[0], four_in[1],four_in[2], four_in[3]);
and(three_out[1], or_allpairs, nand_0123);

endmodule


module test;
    reg  [3:0] in_test;
    wire [2:0] out_test;
    countones dut(in_test, out_test);
    initial begin
        in_test = 4'b0000; #5 $display("in=0000 -> out=%h%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0001; #5 $display("in=0001 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0011; #5 $display("in=0011 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0111; #5 $display("in=0111 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b1111; #5 $display("in=1111 -> out=%h",
out_test[2],out_test[1], out_test[0]);
    end
endmodule
```

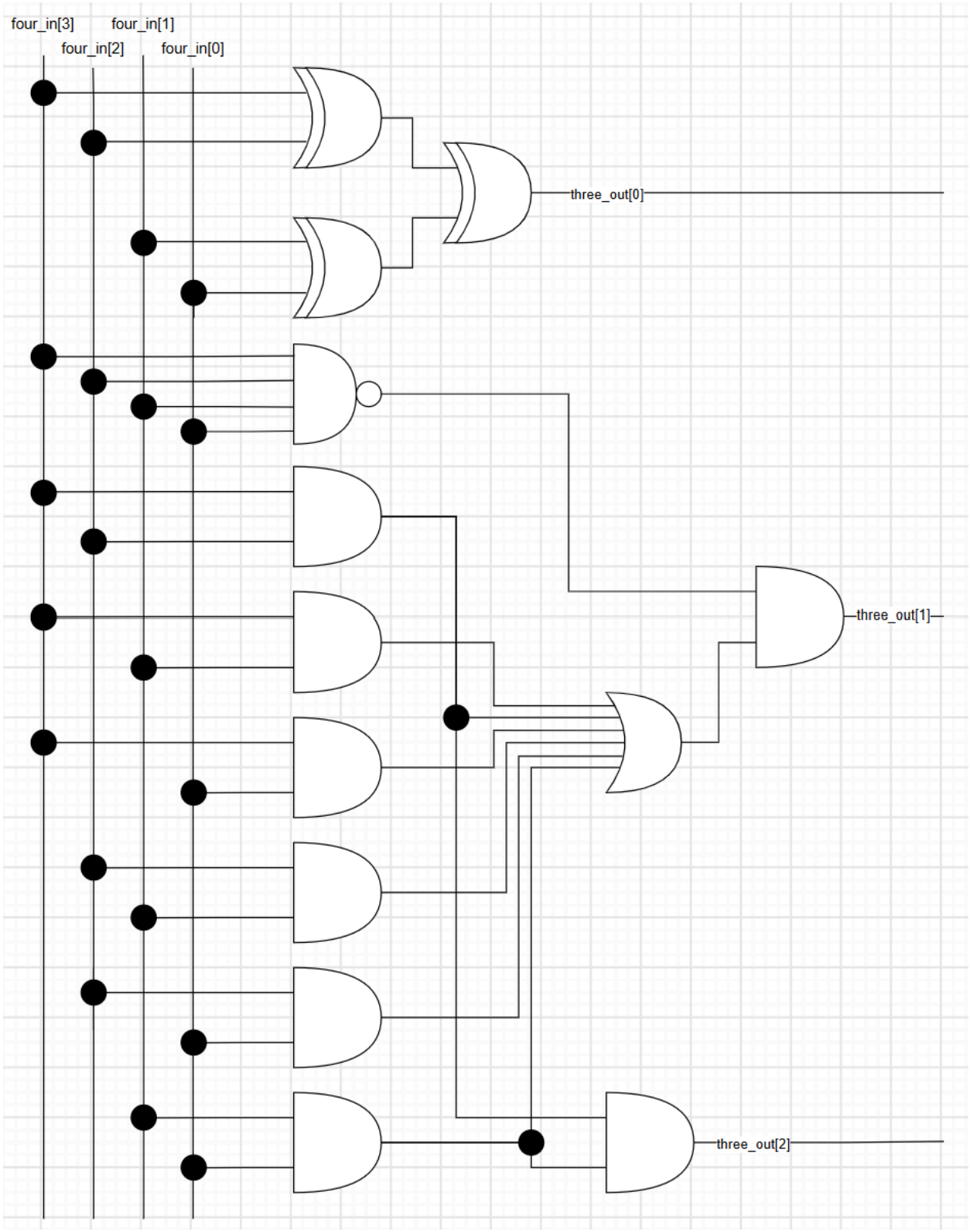
Used logicly demo (https://logic.ly/demo/) to test out logic first:



Partial submission (issues with handling of 571 midterm, wanting to get something submitted)

Shani “Dill” De Silva, U44033199

Leading to the logic schematic below, drawn on draw.io:



ADD SYNTHESIS MESSAGES

b. Write a behavioral Verilog description of the design that will synthesize.

(used https://coddy.tech/playground/verilog to write and quick-test)
Module code and testbench:

```
module countones (
    input  [3:0] four_in,
    output reg [2:0] three_out
);

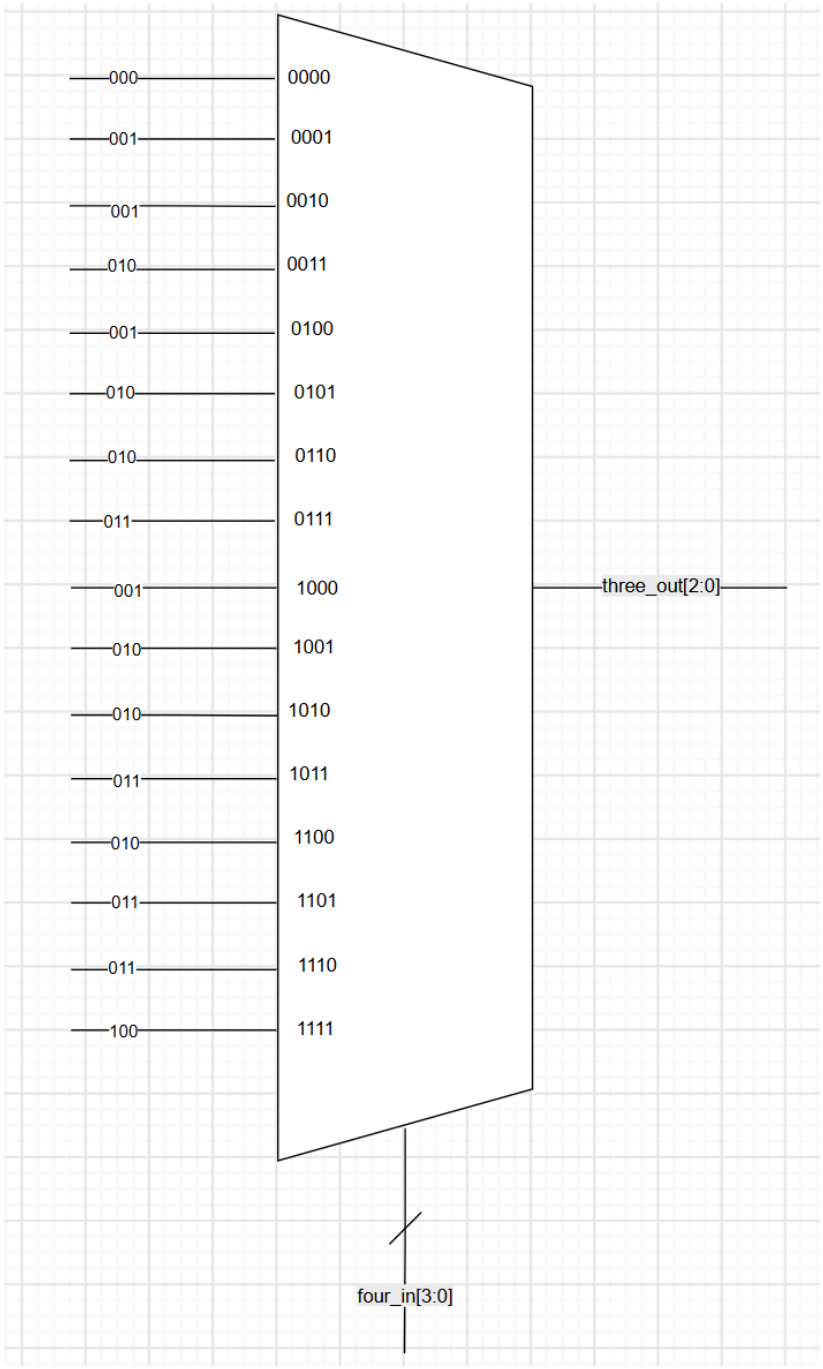
always @* begin
    case (four_in)
        4'b0001, 4'b0010, 4'b0100, 4'b1000: three_out = 3'b001;
        4'b1110, 4'b1101, 4'b1011, 4'b0111: three_out = 3'b011;
        4'b0000: three_out = 3'b000;
        4'b1111: three_out = 3'b100;
        default: three_out = 3'b010;
    endcase
end
endmodule

module test;
    reg  [3:0] in_test;
    wire [2:0] out_test;
    countones dut(in_test, out_test);
    initial begin
        in_test = 4'b0000; #5 $display("in=0000 -> out=%h%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0001; #5 $display("in=0001 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0011; #5 $display("in=0011 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0111; #5 $display("in=0111 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b1111; #5 $display("in=1111 -> out=%h",
out_test[2],out_test[1], out_test[0]);
    end
endmodule
```

Below is the logic schematic, drawn on draw.io:

Partial submission (issues with handling of 571 midterm, wanting to get something submitted)

Shani “Dill” De Silva, U44033199



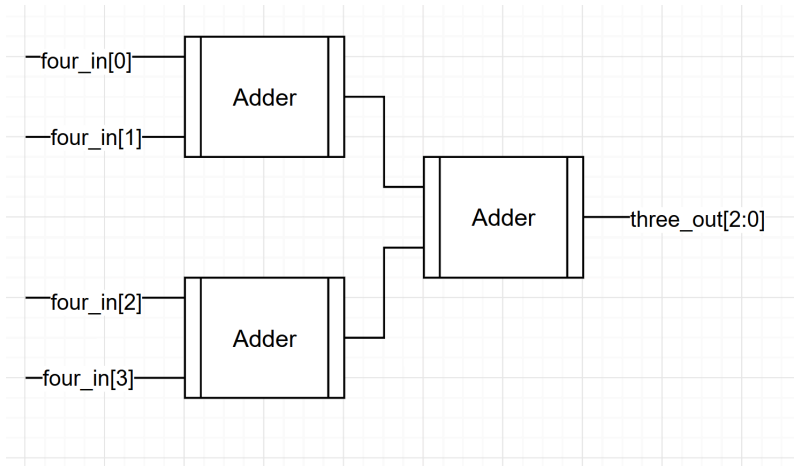
ADD SYNTHESIS MESSAGES

c. Write a behavioral Verilog description of the design that synthesizes AND the resultant circuit is different than part b.

```
module countones (
    input  [3:0] four_in,
    output reg [2:0] three_out
);
    always @(*) begin
        three_out = four_in[0] + four_in[1] + four_in[2] + four_in[3];
    end
endmodule


module test;
    reg  [3:0] in_test;
    wire [2:0] out_test;
    countones dut(in_test, out_test);
    initial begin
        in_test = 4'b0000; #5 $display("in=0000 -> out=%h%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0001; #5 $display("in=0001 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0011; #5 $display("in=0011 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0111; #5 $display("in=0111 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b1111; #5 $display("in=1111 -> out=%h",
out_test[2],out_test[1], out_test[0]);
    end
endmodule
```

Logic schematic below, drawn with draw.io:



ADD SYNTHESIS MESSAGES

d. Write a Verilog description that WILL NOT synthesize. Show the error messages/result that indicates that it will not synthesize. Explain why these occur.

DO THIS

5. Consider a 4-bit counter. This will be a sequential logic circuit (i.e. FSM). The circuit should have a reset signal. This should set it to the initial output state of 4'b0000. The only other input should be a clock signal. Each clock tick should advance the counter. The counter should reset after 4'b1111.

Provide verilog code, schematics, and relevant synthesis messages for all four parts.

a. Write a structural Verilog description of the design that synthesizes.

DO THIS

b. Write a behavioral Verilog description of the design that will synthesize.

DO THIS

c. Write a behavioral Verilog description of the design that synthesizes AND the resultant circuit is different than part b.

DO THIS

d. Write a Verilog description that WILL NOT synthesize. Show the error messages/result that indicates that it will not synthesize. Explain why these occur.

DO THIS